워치 · 스탑워치 통합 설계

Basys3 기반 디지털 시계 및 랩타임 스탑워치



팀원 박재균 · 조수영

발표일 2026.10.18

Contents

01 개요

02 핵심결과 및 역할분담

03 전체 구조

04 설계 상세
- 트러블슈팅

05 시뮬레이션 시나리오

06 시뮬레이션 결과

07 구현 · 동작 검증

08 결론 및 고찰

프로젝트 개요

설계 목적

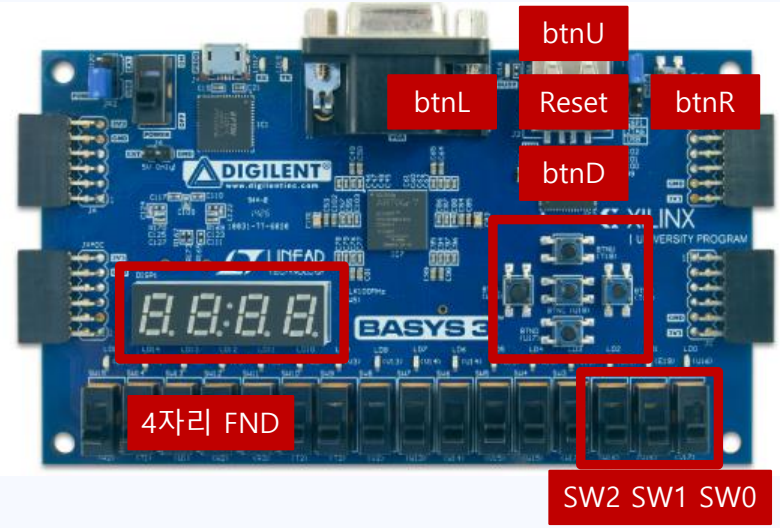
- 독립적으로 작동하는 stopwatch/watch 설계
- 필요한 기능의 흐름도를 정의 , 모듈별 담당 기능으로 세분화
- 각자 설계 · 검증한 모듈을 하나로 통합하는 과정을 학습

모듈화 이유

- **복잡도 분리** · 기능별 분할 → 오류 위치 파악 용이
- **병렬 설계** · 신호 선정의 → 동시 설계
- **재사용** · 카운터 반복 배치 → 코드량 감소

구분	주요 기능	구현 기준
Watch	일반 계수, 자리 선택, 값 수정	선택한 자리만 변경
Stopwatch	실행 · 정지, 초기화, 업다운, 랩 저장	랩 표시 중에도 내부 계수 유지
공통 표시	모드와 시간 단위 선택, 4자리 FND	SW 선택과 표시 데이터 일치

사용 하드웨어와 입출력



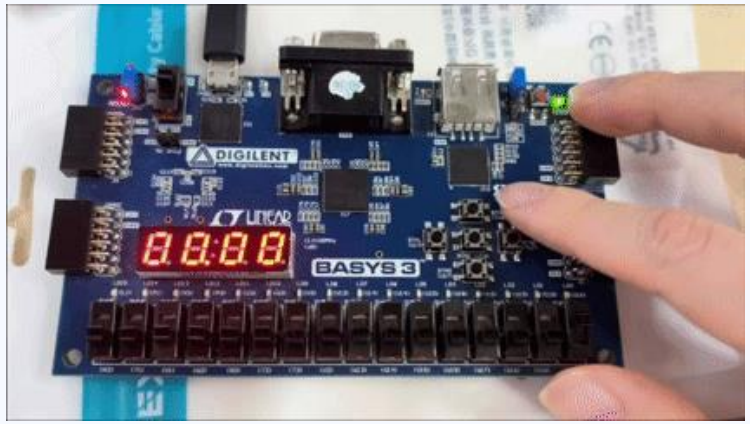
Basys3 입출력 위치

보드 Basys3 (Artix-7) · 100 MHz

버튼	시계 수정 중	스톱워치 화면
btnR	오른쪽 자리	시작 / 일시정지
btnL	왼쪽 자리	리셋 (정지 시)
btnU	선택 자리 +1	업 / 다운 전환
btnD	선택 자리 −1	랩 저장
Reset	전체 초기화 (rst)	전체 초기화 (rst)

스위치	0	1
SW0	시계 화면	스톱워치 화면
SW1	sec. msec	hour. min
SW2	일반	시계 시간 수정

결과 요약 및 역할 분담



자리별 시간 수정 및 수정 위치 표시

- 선택한 자리만 변경 및 변경 중인 현재 자리수 점멸 표시
- 수정 모드 해제 후 변경한 시간부터 카운트 시작

Laptime 기능 추가

- 내부 카운트는 유지하고 , 화면에는 저장한 시점의 값 표시
- laptime 버튼 재입력 시 저장값 갱신

CONTROL UNIT

박재균

제어 유닛

- stopwatch_control_unit
- watch_control_unit
- 디바운스 · 버튼 분배
- 단위 시뮬레이션 U1–U4

DATAPATH

조수영

데이터패스

- stopwatch_datapath
- watch_datapath
- 시간 카운터 · 랩 저장 · 표시 MUX
- fnd_controller

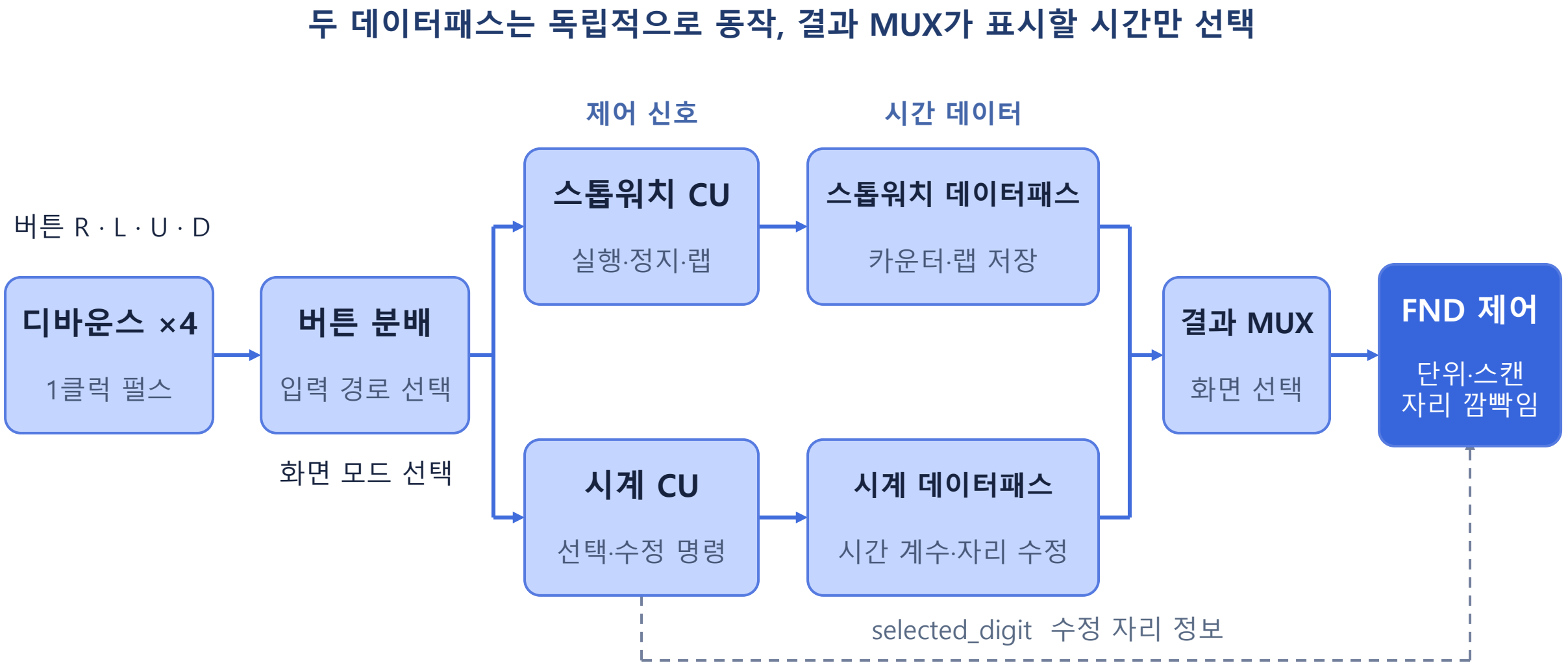
TEAM

공동

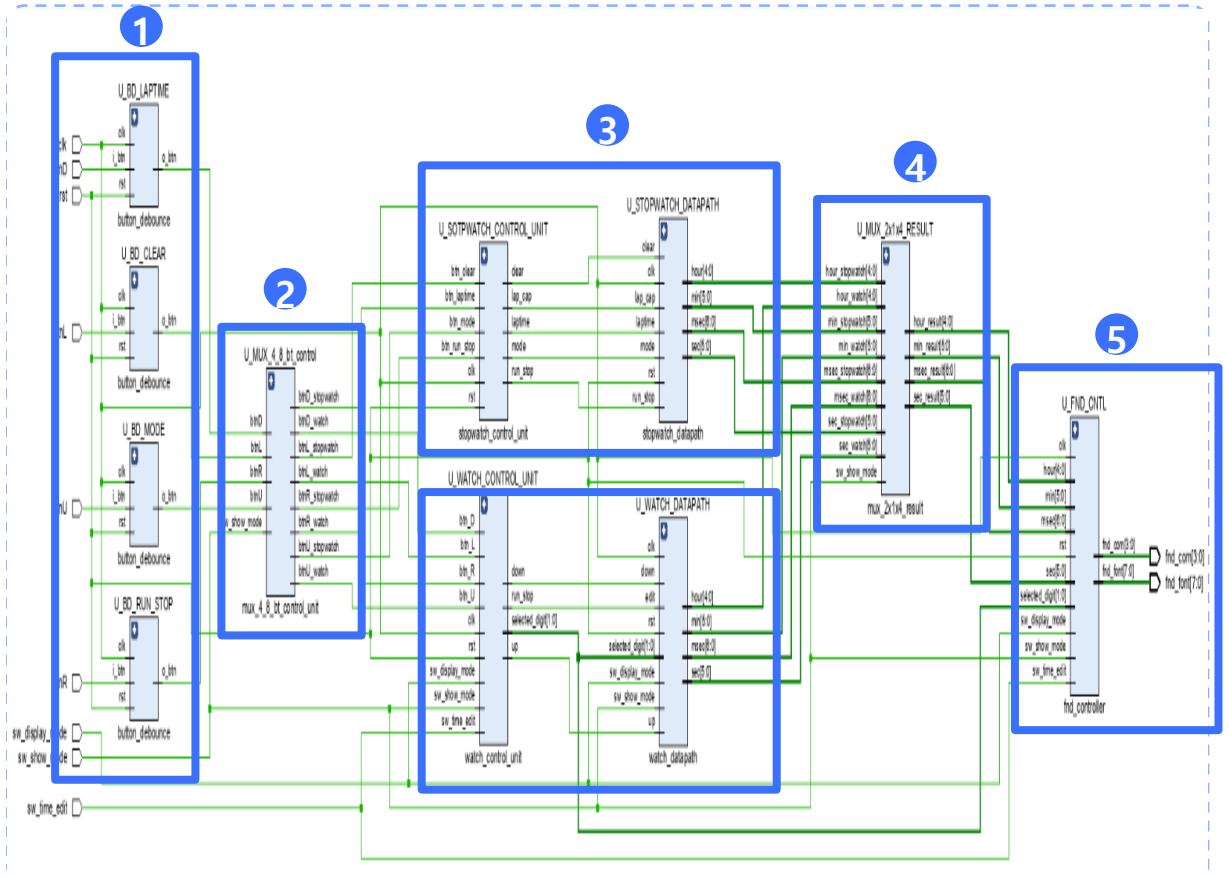
공동 작업

- Top 연결
- 통합 시뮬레이션 S1–S5
- 보드 검증
- 보고서 · 발표 준비

전체 구조


두 데이터패스는 독립적으로 동작, 결과 MUX가 표시할 시간만 선택
제어 신호
시간 데이터
버튼 R · L · U · D
디바운스 ×4
1클럭 펄스
버튼 분배
입력 경로 선택
화면 모드 선택
스톱워치 CU
실행·정지·랩
스톱워치 데이터패스
카운터·랩 저장
시계 CU
선택·수정 명령
시계 데이터패스
시간 계수·자리 수정
결과 MUX
화면 선택
FND 제어
단위·스캔
자리 깜빡임
selected_digit 수정 자리 정보

최상위 회로도



1. **디바운스 4개**
   버튼마다 button_debounce 1개

2. **버튼 분배**
   AND 게이트 8개, SW1 · ~SW1로 분기

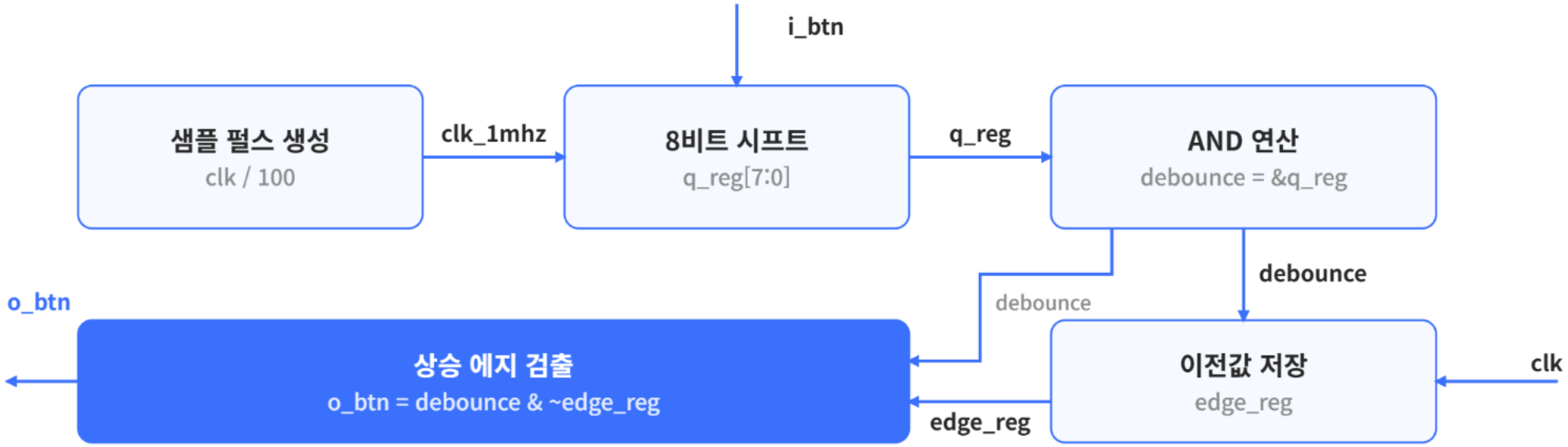
3. **stopwatch | watch**
   스톱워치 · 시계 블록 간 연결 없음

4. **출력 선택 MUX**
   선택 값 + selected_digit 전달

5. **7 segements 출력**
   선택 값 + selected_digit 전달

버튼 디바운스

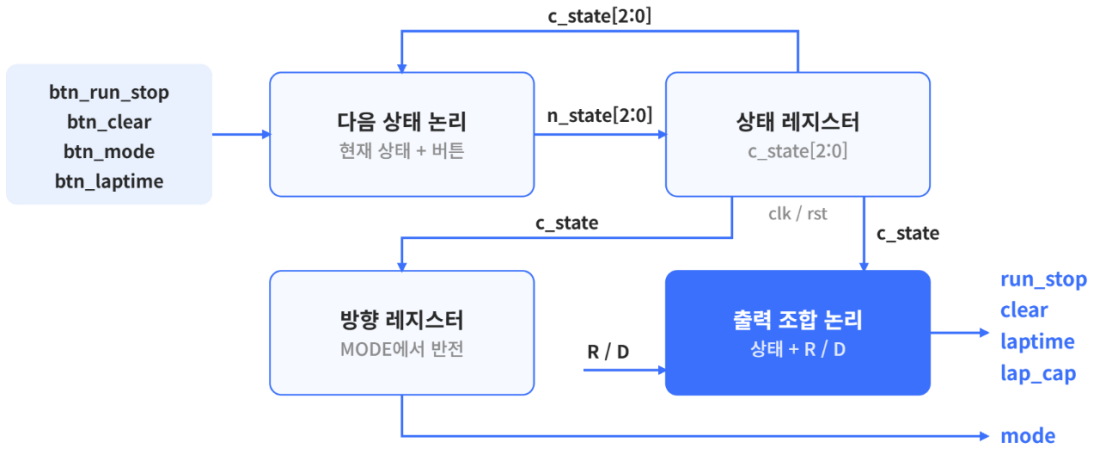
입력이 8회 연속 HIGH로 샘플링되면 한 번의 버튼 이벤트를 만든다



샘플 주기 1 MHz 샘플: 1 µs 간격 · 기본 clk 100 MHz

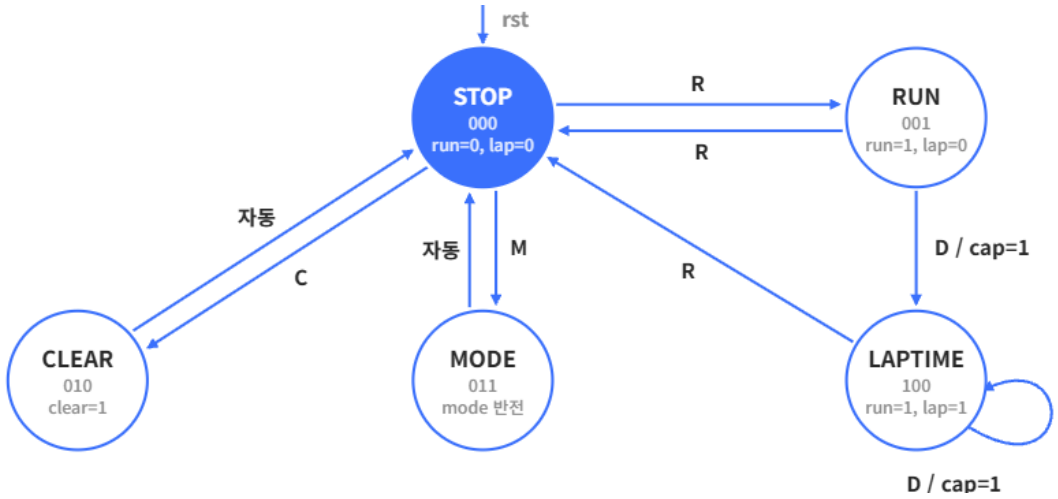
출력 8회 연속 HIGH 확인 → 상승 에지에서 1 clk 폭 펄스

스톱워치 제어 유닛



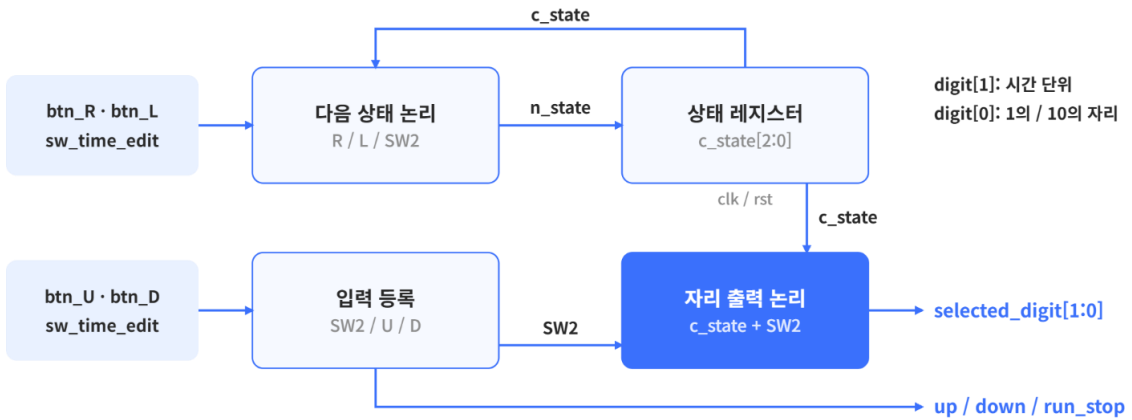
Moore run_stop · clear · laptime ← 현재 상태

Mealy lap_cap = (RUN | LAPTIME) & !R & D



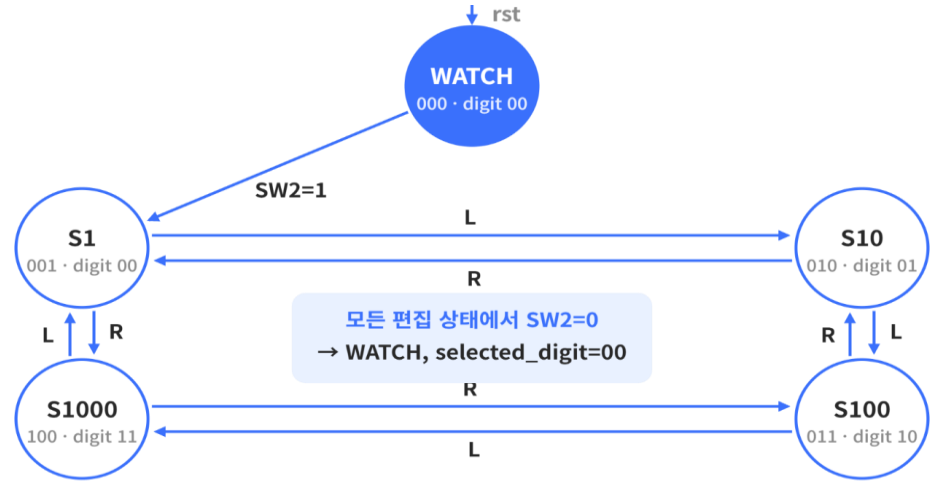
R 실행 · 정지 **C** 초기화 **M** 방향 변경 **D** 랩 저장 · 나머지 입력은 현재 상태 유지

시계 제어 유닛



Mealy selected_digit ← 상태 + SW2

연결 run_stop → 데이터패스의 edit



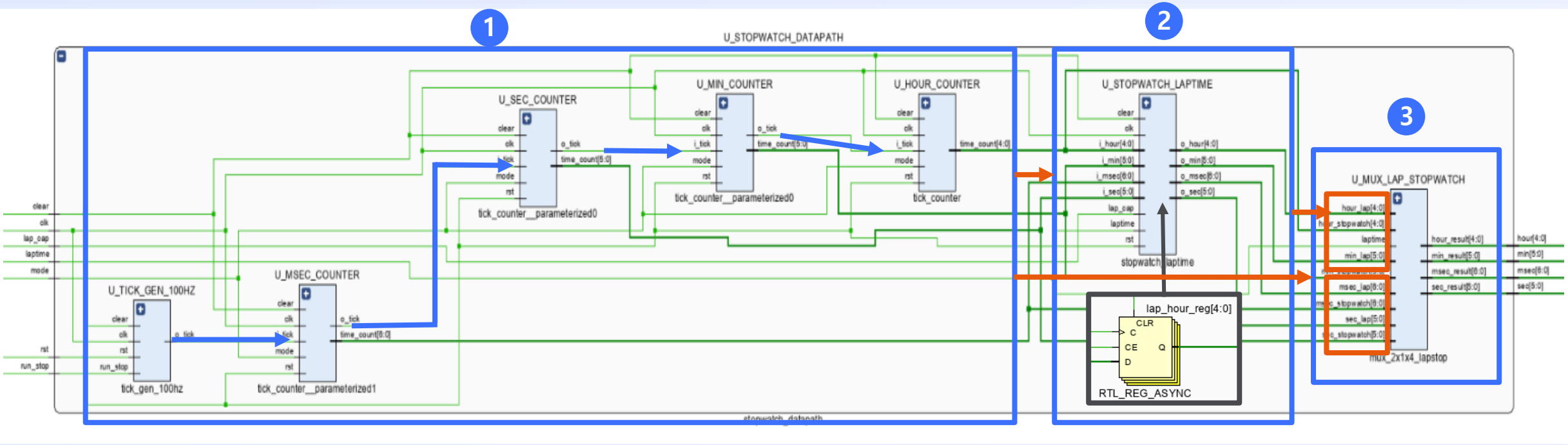
digit은 SW2=1 기준 · R / L 동시 입력 시 **R 우선** ·
U / D는 자리를 바꾸지 않음

stopwatch_datapath

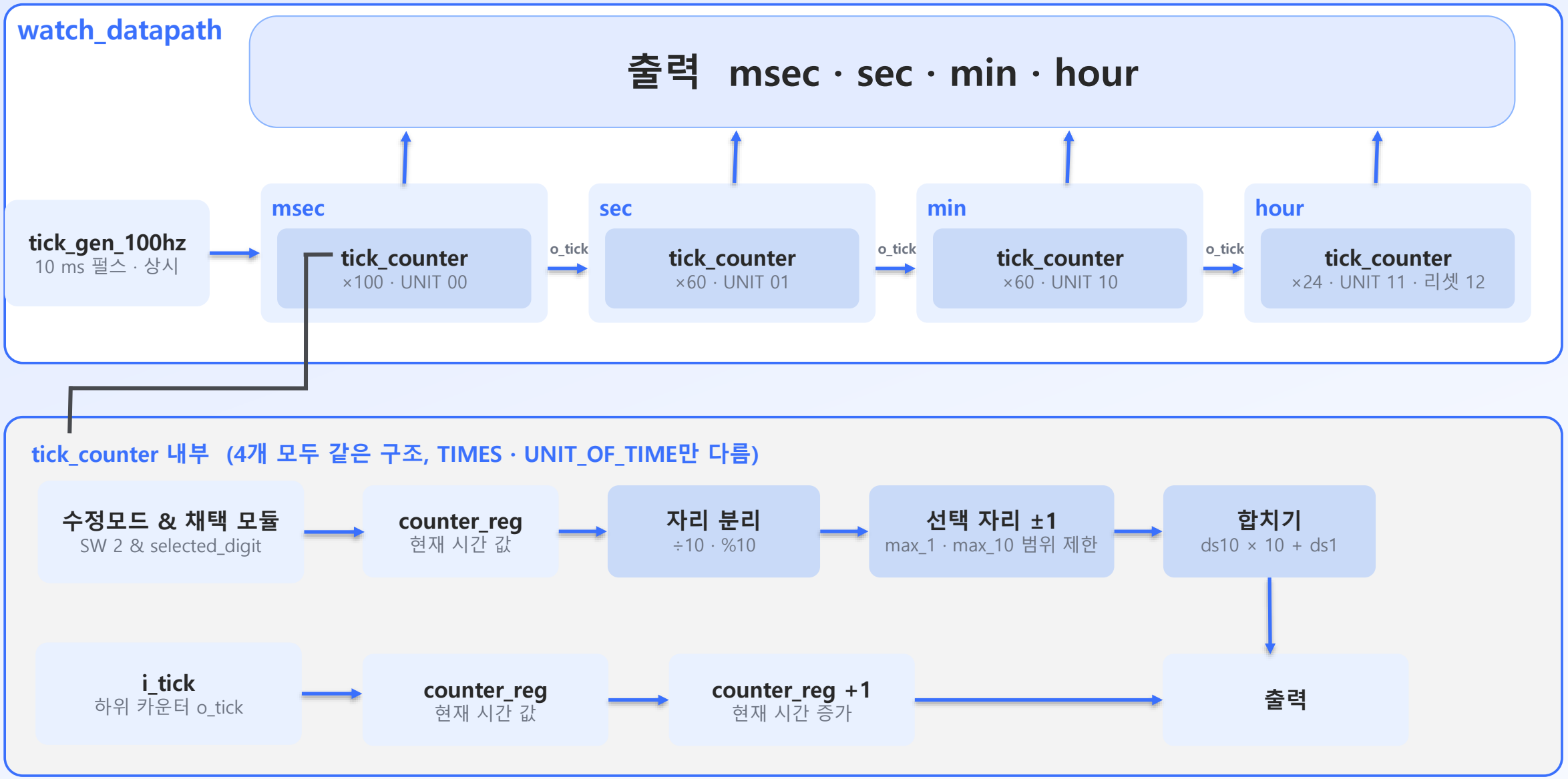
1 tick 연결
tick_gen → tick_counter 4개 직렬 연결

2 laptime 레지스터
레지스터 4개

3 출력 MUX
mux_2x1x4_lapstop만 출력 구동



watch_datapath 다이어그램



watch_datapath - tick_counter 범용 모듈 설계

TIMES 하나만 받아 최댓값과 자리별 한계를 계산 → 범위가 다른 msec · sec · min · hour를 같은 모듈로 처리

① 파라미터 → 로컬 파라미터

```
tick_counter_watch #(
    .TIMES       (24),
    .BIT_WIDTH   (HOUR_BIT_WIDTH),
    .UNIT_OF_TIME(2'b11)
```

```
localparam max = TIMES - 1;
localparam max_1 = max % 10;
localparam max_10 = max / 10;
```

② 인스턴스별 자동 계산 값

단위	TIMES(카운트횟수)	max	max_10	max_1
msec	100	99	9	9
sec	60	59	5	9
min	60	59	5	9
hour	**24**	**23**	**2**	**3**

③ 1의 자리 범위

10의 자리 = max_10 → 0 ~ max_1
그 외 → 0 ~ 9

예) hour 2x : 1의 자리 0 ~ 3 (23 다음 20)

④ 10의 자리 범위

1의 자리 > max_1 → 0 ~ max_10 − 1
그 외 → 0 ~ max_10

예) hour 19 : 10의 자리 0 ~ 1 (29 불가)

같은 모듈 × 4 인스턴스 · TIMES만 바꿔 범위가 다른 단위에 재사용

FND_controller

100 MHz
시스템 클럭

→

2 kHz 분주
÷50,000

→

ds_sel 0→7
counter_8

→

자리 · 값 선택
MUX

→

decoder | bcd
7segments 출력

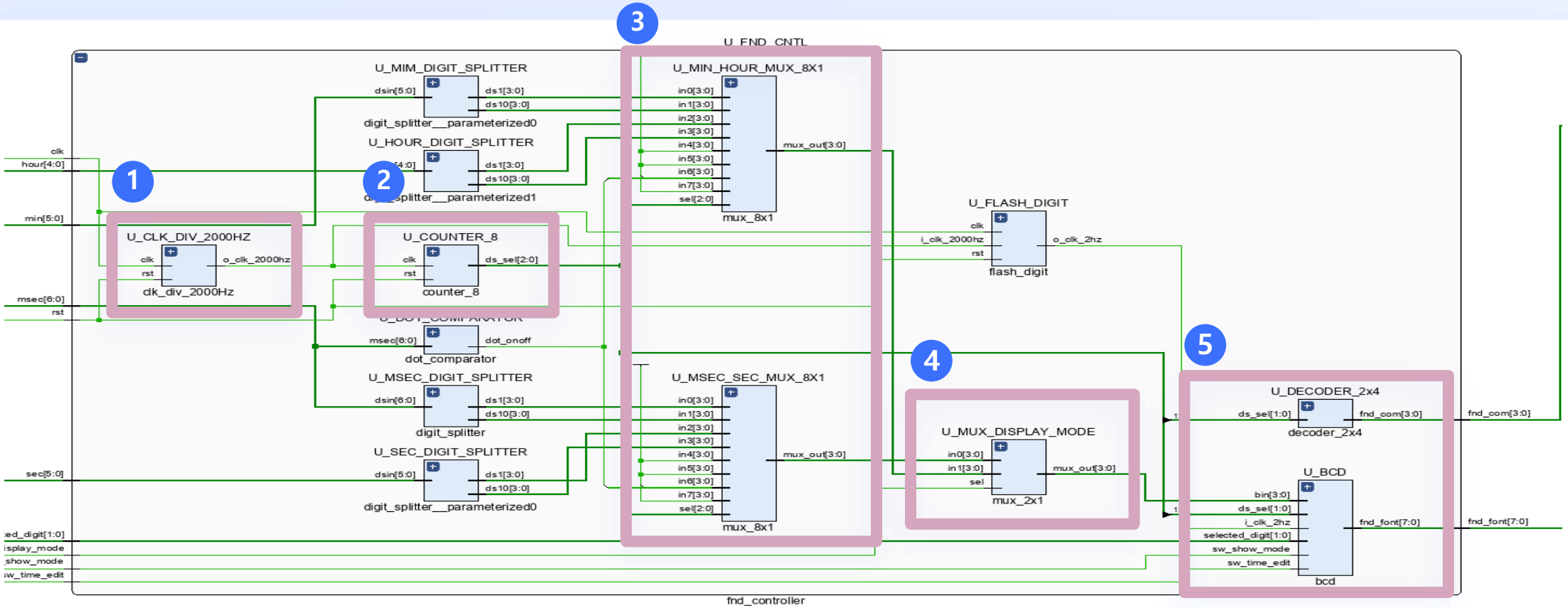
ds_sel	fnd_com	SW0=0	SW0=1
0	1110	1/100초 1의 자리	분 1의 자리
1	1101	1/100초 10의 자리	분 10의 자리
2	1011	초 1의 자리	시 1의 자리
3	0111	초 10의 자리	시 10의 자리
4 · 5 · 7	—	꺼짐	꺼짐
6	1011	가운데 점	가운데 점

잔상 효과
슬롯 0.5 ms, 프레임 4 ms (250 Hz)

active-low
fnd_com · fnd_font 0일 때 점등

수정 위치 깜빡임
2hz 깜빡임 반복

fnd_controller 회로도



1 분주 클럭
clk_div → counter_8 · flash_digit 클럭

2 출력 위치
숫자4개 + dot표시 4개 -> ds_sel 출력

3 MUX 2단
1,10 자리로 쪼개진 데이터중 선택

4 출력 데이터 결정 mux
min:hour or sec:msec

5 fnd 출력
decoder: segment on/off
bcd: 숫자 -> segment 용 출력변환
수정 커서 깜빡임 로직

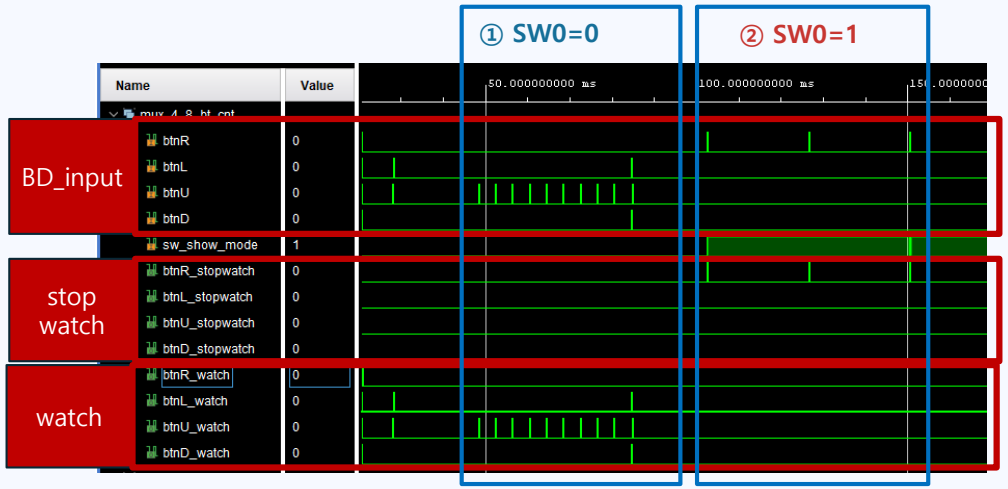
단위 시뮬레이션 시나리오

모듈 단위로 입력 → 기대 출력을 먼저 확인한 뒤 통합 시나리오(S1–S5)로 넘어간다

대상	입력 시나리오	기대 출력 · 관찰 신호
버튼 디바운스	i_btn을 HIGH로 유지 샘플 펄스 8회 관찰	8번째 HIGH 샘플에서 o_btn 발생 누르는 동안 추가 펄스 없음
버튼 분배	SW0=0에서 버튼 입력 SW0=1로 전환 후 재입력	선택한 CU 출력만 입력 추종 비선택 경로는 0
Watch CU	SW2=1, L 연속 입력 U / D 각각 입력	digit 순환 00 → 01 → 10 → 11 → 00 up / down에 입력 전달
Stopwatch CU	랩 표시 중 C / M / D 입력 R로 정지	C / M은 상태 유지, D는 lap_cap R 이후 run_stop=0, laptime=0

시뮬레이션: 버튼 디바운스 + 분배기

1. 분배기



① SW0=0 · Watch 선택

Watch 출력이 입력 펄스를 추종

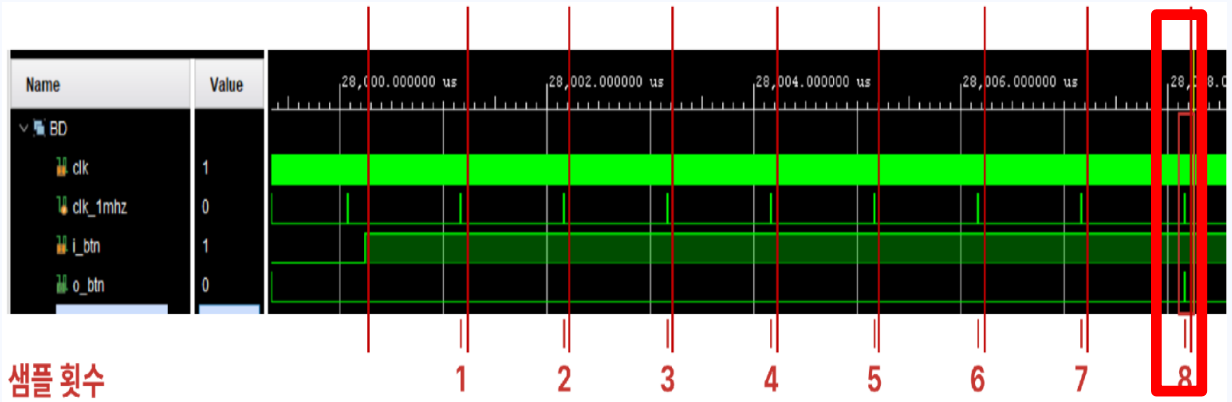
Stopwatch 출력은 0

② SW0=1 · Stopwatch 선택

Stopwatch 출력이 입력 펄스를 추종

Watch 출력은 0

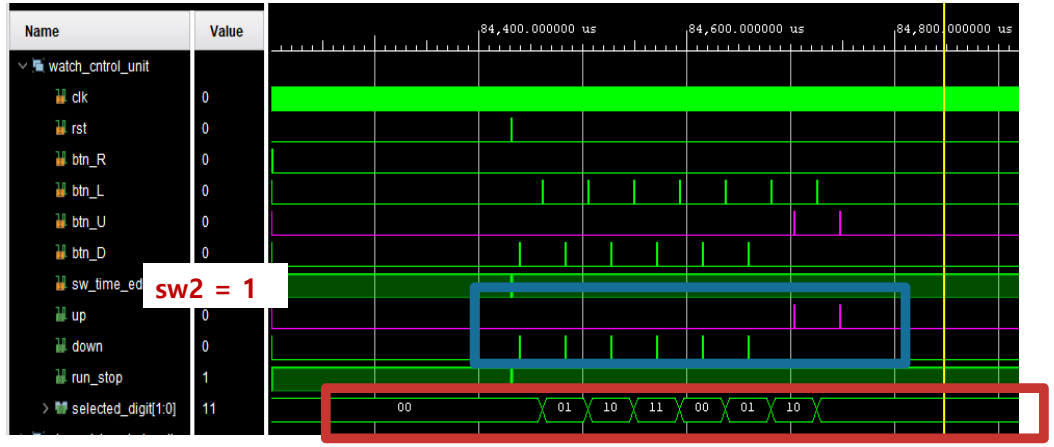
2. 버튼 디바운스



i_btn=1 유지 1 MHz 샘플 8회 관찰

시뮬레이션: Watch+ stopwatch 컨트롤 유닛

1. watch 컨트롤 유닛



입력 조건

SW2=1 유지

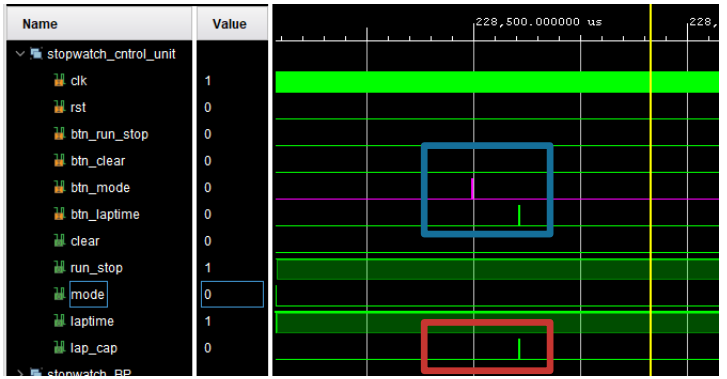
L 반복 입력, U / D 각각 입력

관찰 결과

L마다 선택 자리 순환

00 → 01 → 10 → 11 → 00

U / D와 up / down 대응



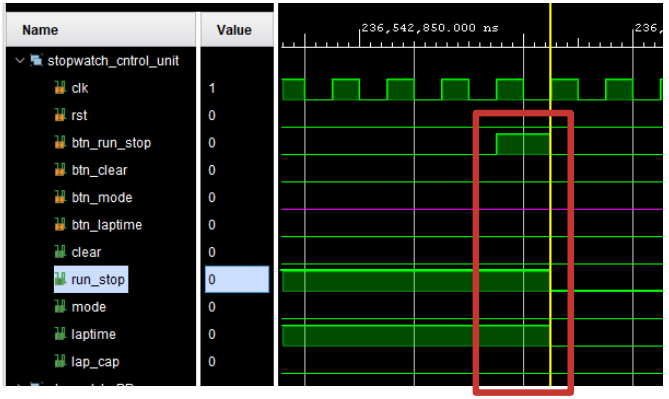
2. Stopwatch 랩동작

LAPTIME 중

C → M → D

C / M: clear=0, mode 유지 run_stop=1, laptime=1

D: lap_cap 펄스 발생



3. Stopwatch 랩 종료

R 입력 → STOP 전환

run_stop: 1 → 0

laptime: 1 → 0

lap_cap: 0 유지

스톱워치 · 랩타임 시나리오

S1 스탑 후 모드 변경

입력 / 동작	기대 결과
RUN 상태에서 STOP 입력	카운트 정지 FND는 정지 시점의 시간 유지
STOP 상태에서 MODE 변경	정지 상태를 유지하며 카운트 방향 변경
btnR (RUN) 입력	모드 변경 전과 반대 방향으로 카운트 시작
RUN 상태 유지	변경된 방향으로 카운트하며 FND 표시 갱신

STOP → MODE 변경 → RUN 순서와 카운트 방향

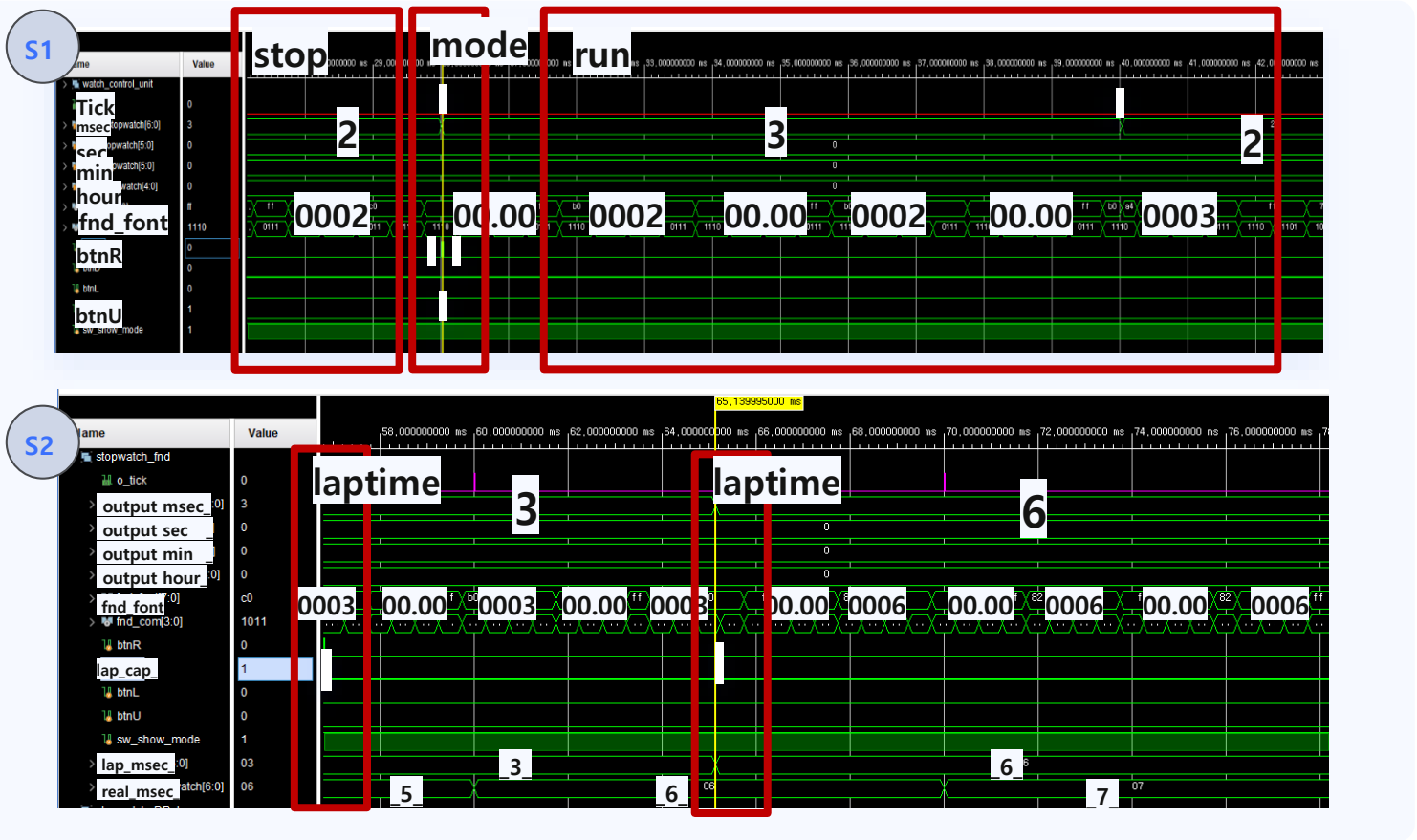
S2 랩타임

입력 / 동작	기대 결과
RUN 상태	내부 real time 정상 카운트 FND는 real time 표시
LAP 버튼 첫 입력	내부 real time은 계속 카운트 FND는 입력 시점의 시간 고정
LAP 표시 유지	내부 real time은 계속 카운트 FND는 저장된 랩타임 유지
LAP 버튼 재입력	FND가 현재 real time 표시로 복귀 이후 실시간으로 갱신

내부 real time과 FND 표시값 비교

시뮬레이션: 모드 변경과 랩타임

모드 변경은 stop 상태를 거치며, 랩타임은 표시만 변경



① S1 · 스탑 후 모드 변경

동작순서

1. stop 상태로 진입
2. mode 변경
3. run 상태로 전환

동작결과

모드 변경 전과 반대 방향 카운트 시작

② S2 · 랩타임

동작순서

1. 카운트 중 lap 버튼 입력
2. lap 버튼 재입력

동작결과

real time은 정상적으로 카운트
FND는 lap 버튼 누른 시점 시간 출력

watch 수정· 동시 카운트 시나리오

S3 시간 수정, 수정 커서부분 점멸 확인

입력	기대 결과
리셋	12:00, 시간 자동 진행
sw2 = 1	**수정** on -> conter값 증가 x
btnR	1000의 자리로 이동 (경계값)
sw1 = 1	hour:min = 12:00
btnU * 2	**hour: 12 ->22->02**
btnR	100의 자리로 이동
btnD * 3	**hour: 02 ->01 -> 00->09**
btnL	1000의 자리로 이동
btnD	**hour: 09 -> 19**
sw2 = 0	수정된 값에서 증가

S4 stopwatch / watch 동시카운트

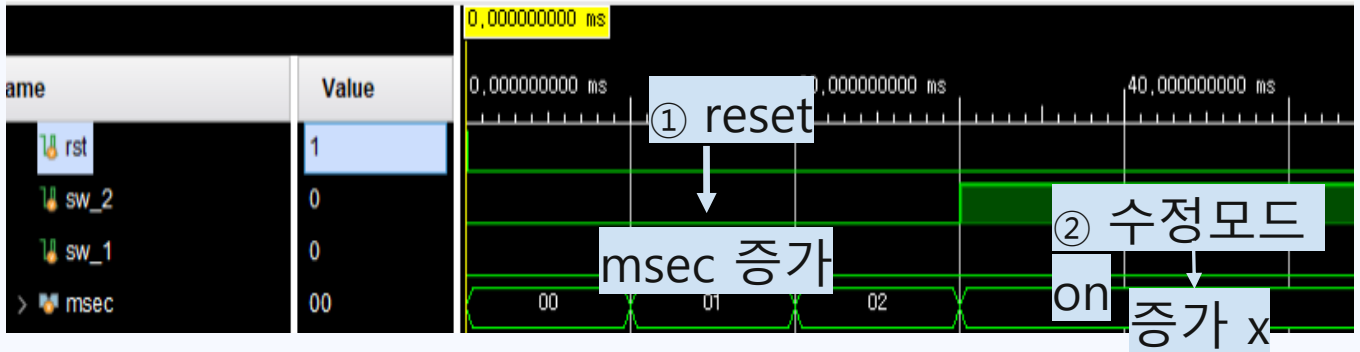
입력	기대 결과
sw0 = 0	watch
sw0 = 1	stopwatch **watch도 카운트 증가**
btnR	RUN
sw0 = 0	watch **stopwatch도 카운트 증가**

S3 시간 수정 · 깜빡임

시나리오 3 - 시뮬레이션

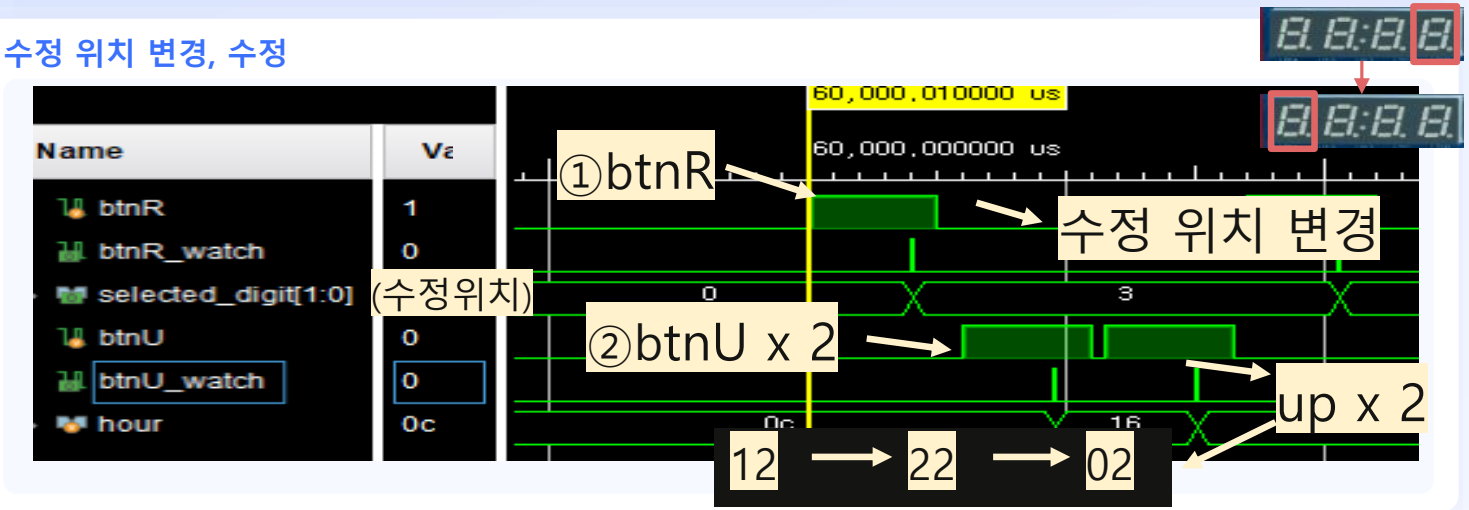
시간 수정 => 정상 동작 확인

reset -> msec 증가 | sw2 = 1 -> 증가 x



→ 수정모드시 -> 시간 증가 x

수정 위치 변경, 수정



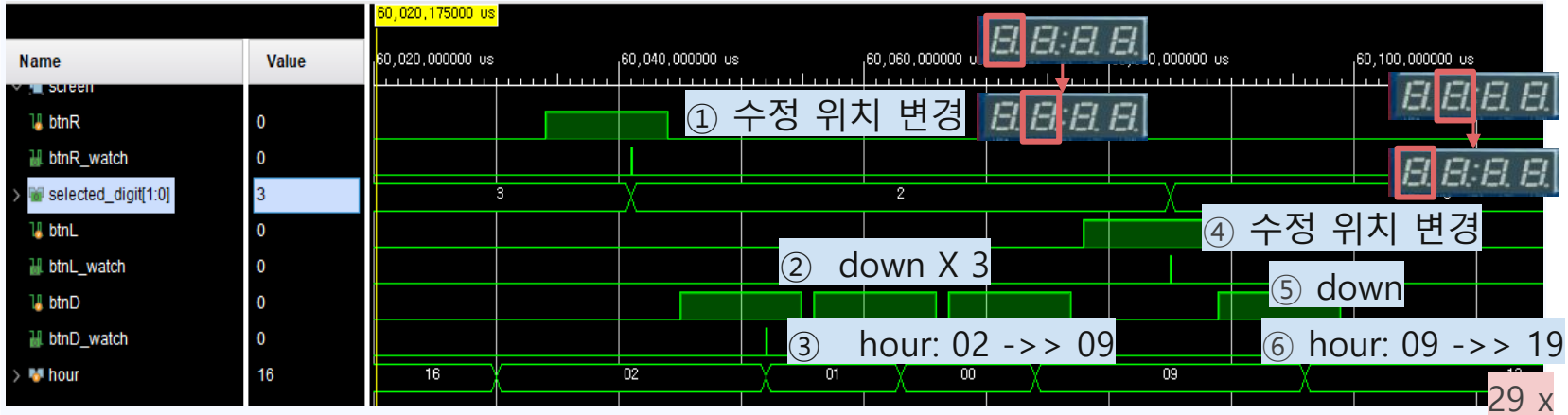
→ 수정 커서이동, UP/DOWN 정상 작동

S3 시간 수정 · 깜빡임

시나리오 3 - 시뮬레이션

엣지 케이스 증감, 모드 전환시 카운트=> 정상 동작 확인

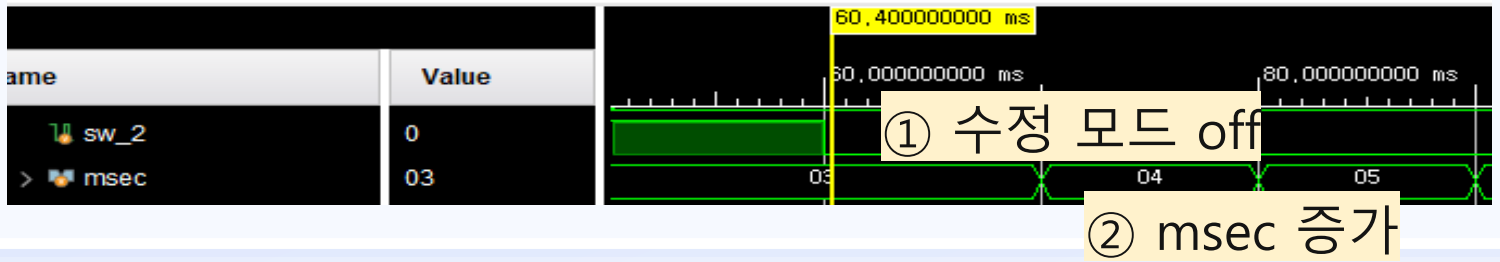
hour 수정



엣지케이스
1의 자리 down: 00 -> 09
10의 자리 up: 09 -> 19

=>정상 작동

sw 2 =0 -> 시간 증가



수정모드 off -> 카운트 증가

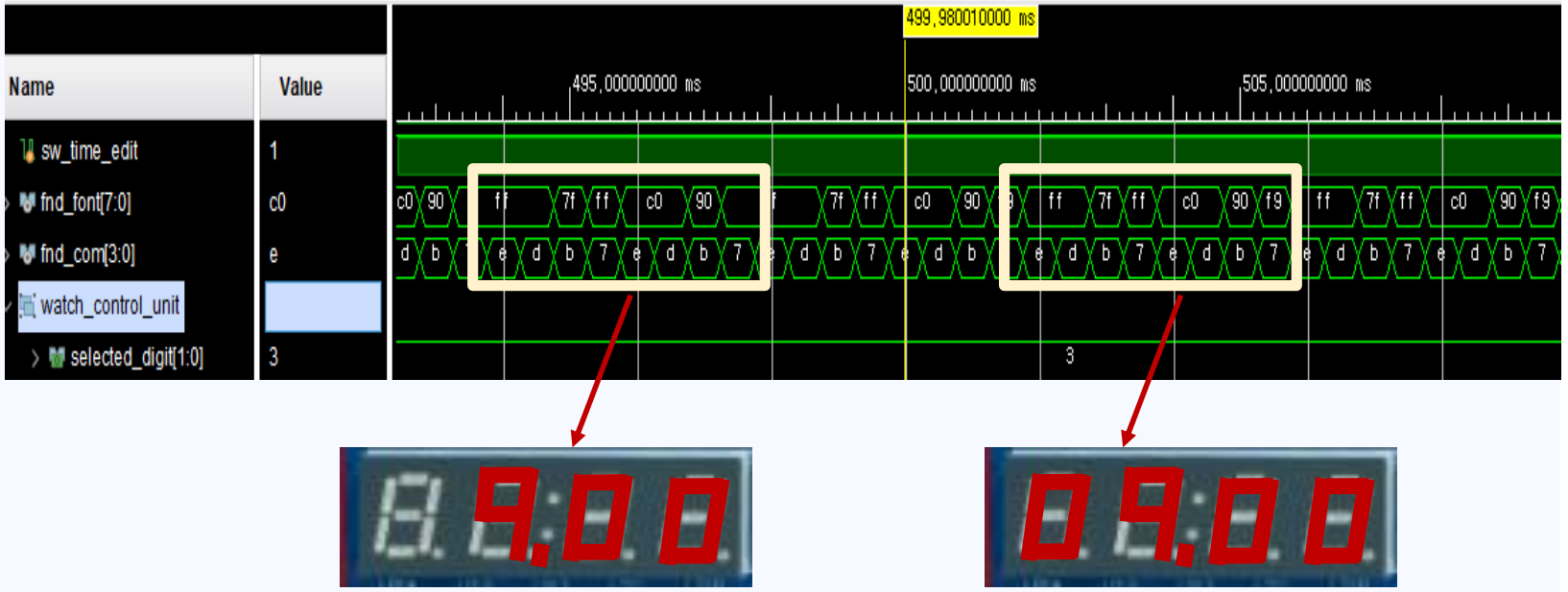
=> 정상 작동

S3 시간 수정 · 깜빡임

시나리오 3 - 시뮬레이션

수정 커서 점멸 => 정상 동작 확인

수정 시간 7segments 깜빡임



수정시 수정 커서부분 점멸

=>정상 작동

S4 stopwatch / watch 백그라운드 작동

시나리오 4 - 시뮬레이션

watch, stopwatch 동시 작동 => 정상 동작 확인



stopwatch / watch 모드 변경 => 화면 모드 상관 x 계속 count

경계값 시뮬레이션 시나리오

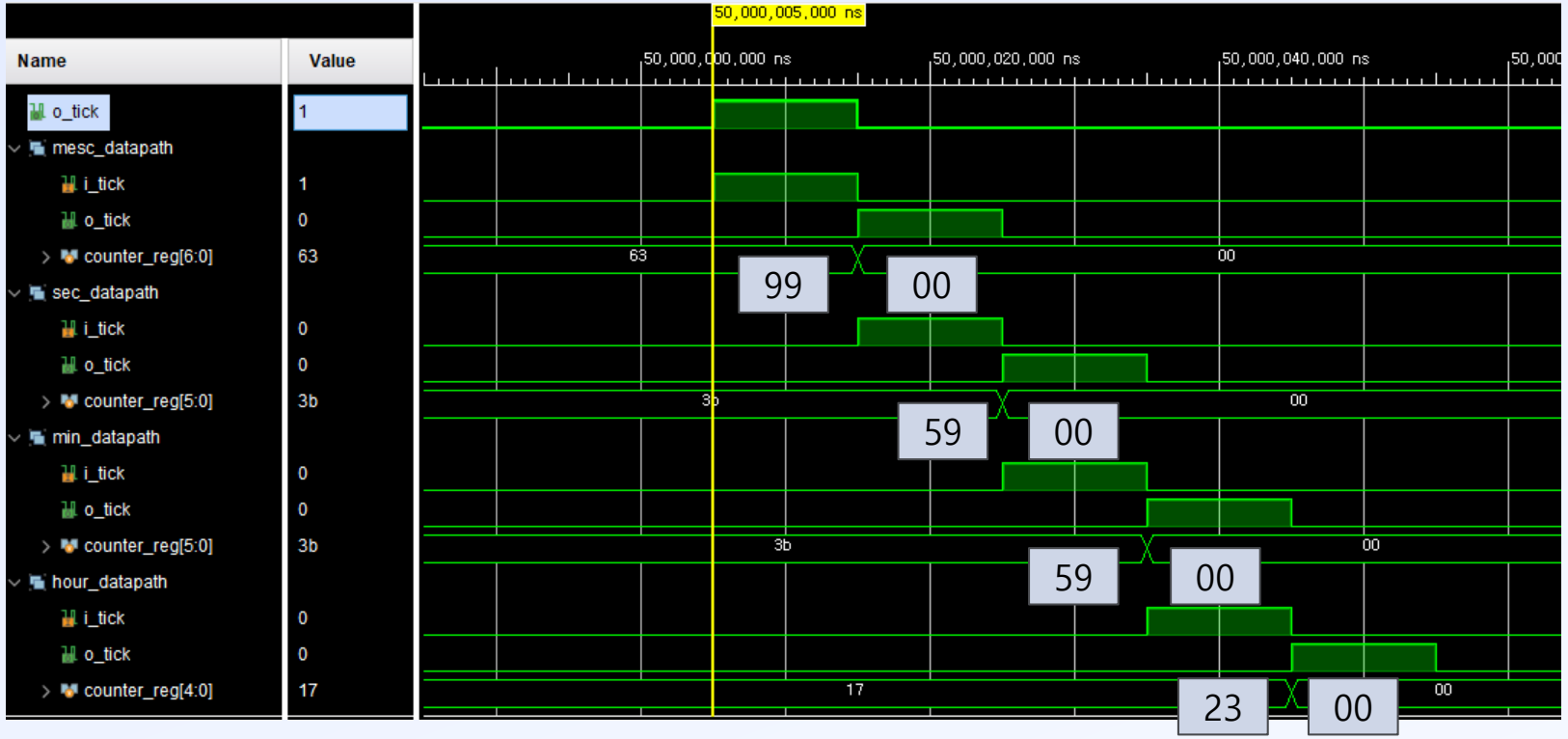
S5 경계값

입력	기대 결과
hour 23,min 59, sec 59,msec 99	
tick_gen의 b_tick = 1	hour: 23-> 00 min: 59->00 sec: 59->00 msec: 99->00

S4 stopwatch / watch 백그라운드 작동

시나리오 5 - 시뮬레이션

경계값 => 정상 동작 확인

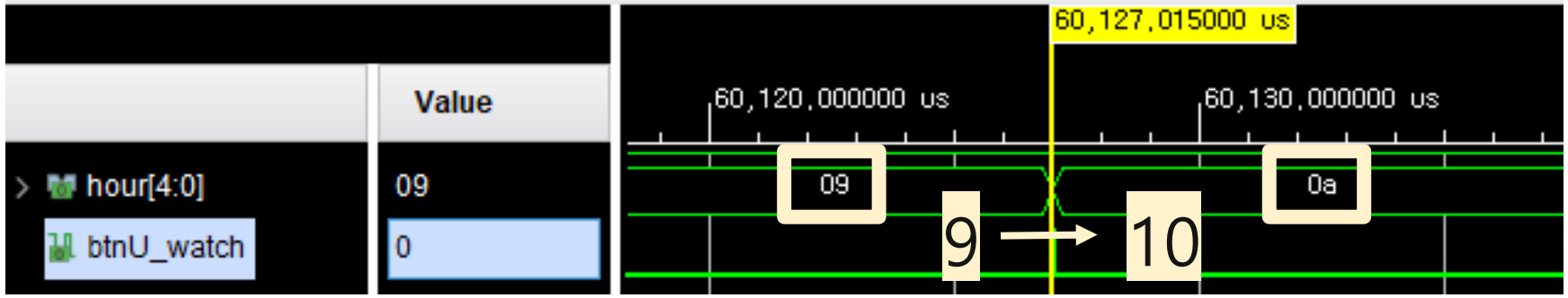


경계값 정상작동

=>정상 작동

TROUBLESHOOTING

1 **현상** 1의 자리 +1 : 09→ 10, **10의 자리까지 변경**



2 **원인** **counter 전체**에 ±1 / ±10 → 자리 올림 · 내림

```
if (up) begin
    counter_reg <= (counter_reg < 7'd111011) ? counter_reg + 1'b1 : 7'd0;
end else begin
    counter_reg <= (counter_reg > 7'd0) ? counter_reg - 1'b1 : 7'd111011;
```

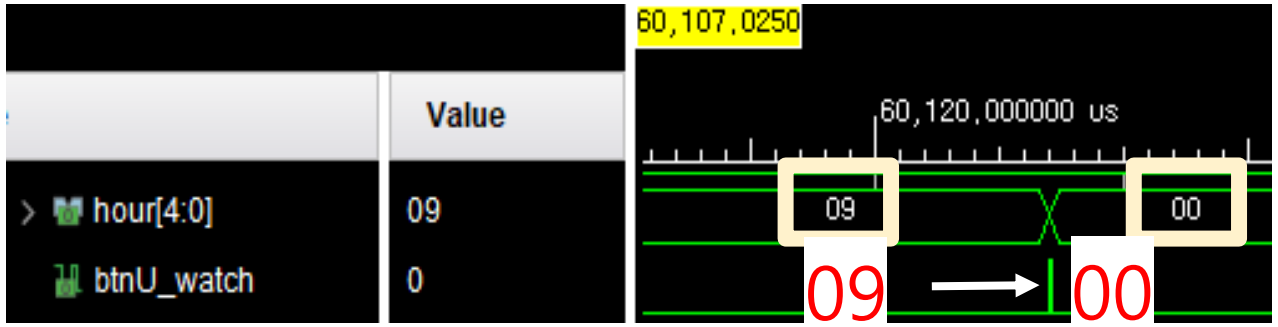
TROUBLE SHOOTING

3 **해결** 1의자리, 10의 자리로 **분리** → 자리별 조건 → 합침

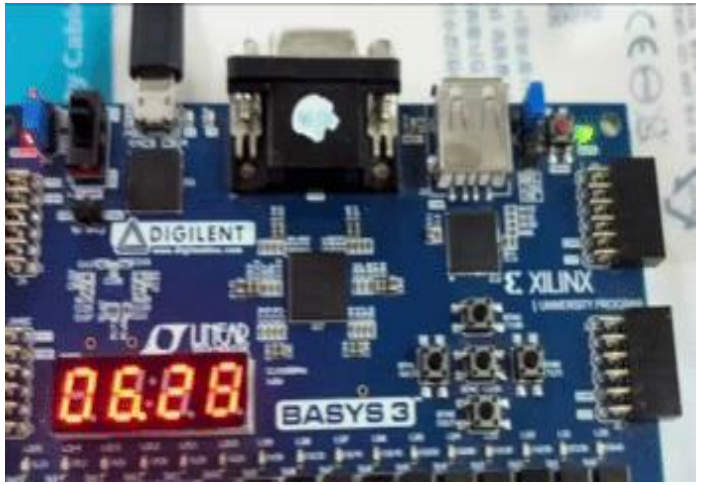
```
counter_next_ds1 = (counter_ds10 == max_10) ? ((counter_ds1 == max_1) ? 0 : counter_ds1 + 1)
                                            : ((counter_ds1 == 9)     ? 0 : counter_ds1 + 1);

counter_next_ds10 = (counter_ds1 > max_1) ? ((counter_ds10 == max_10 - 1) ? 0 : counter_ds10 + 1)
                                          : ((counter_ds10 == max_10)     ? 0 : counter_ds10 + 1);
```

4 **결과** 1의 자리 변경이 10의 자리에 영향 없음 => **정상**동작



실제 보드 동작



시계 hour:min |sec:msec

표시모드 전환

=>정상 작동



수정 커서 깜빡임

수정 & 커서 점멸

=>정상 작동



스톱워치 랩 표시

laptime 기능

=>정상 작동

한계점 및 개선 방향

	LIMITATION	IMPROVEMENT
1	증감 모드 확인 불가	**증감 모드 전환 시 현재 상태 LED 출력**
2	laptime과 stop 구별 불가	**laptime 시 7-segment 점멸 표시**
3	기능 다양성 부족	**타이머 기능 추가**

결론 및 고찰

결론 구현 성과

01

tick_counter 재사용

watch · stopwatch에서 같은 모듈 사용
→ 코드 가독성 증가

02

독립적 카운팅

watch와 stopwatch가 서로 영향 없이
각자 카운트

03

부가 기능 설계

랩타임 · 시간 수정 · 수정 커서 깜빡임

고찰 배운 점 · 개선할 점

1. **기능 흐름 정리 → 모듈 분리**
   상태 판단과 데이터 연산을 서로 다른 모듈로 분리

2. **모듈 연결 시 합의 필요 / 문서화**
   각자 설계한 모듈의 신호 의미 · 상태 조건 · 우선순위

3. **검증 시나리오 확장**
   정상 동작뿐 아니라 다양한 시나리오를 검증할 아이디어 필요

4. **시간 분배**
   설계 목표에 따른 일정 · 시간 분배 필요

THANK YOU

감사합니다

질의응답 · Q&A

